

VeloxQuant-MLX: A KV-Cache Quantization Suite for LLM Inference on Apple Silicon

Rajveer Rathod ¹

¹ Independent researcher

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

When a large language model (LLM) generates text, it stores, for every word it has already processed, two internal vectors — a *key* and a *value* — so that it does not have to recompute them for each new word. This store is called the key–value (KV) cache, and it grows steadily as the conversation or document gets longer. On Apple Silicon Macs, the processor, graphics unit, and neural engine all share a single pool of memory, so this ever-growing cache competes directly with the model itself and with everything else running on the machine. In practice, it is often the KV cache — not the size of the model — that decides how long a document a Mac can handle before it runs out of memory.

VeloxQuant-MLX is an open-source Python library that shrinks this KV cache so that the same Mac can handle longer inputs. It works with models loaded through `mlx_lm`, Apple’s library for running LLMs on its own hardware, and it can be switched on by changing three lines of code; the normal text-generation call is left untouched. The library offers thirty-seven interchangeable compression strategies, so a user can trade a little accuracy for a lot of memory savings, or the reverse, by selecting a different option. These span several families: low-bit *quantization*, *vector quantization* against learned or fixed codebooks, *cross-layer* sharing that exploits redundancy between network depths, and a *token-eviction* family that keeps a constant-size cache by dropping (or merging) the tokens a scoring rule judges least important. Some of the heavy numerical work is implemented as small programs (“kernels”) written for Apple’s Metal graphics interface and compiled on the fly, with a slower but portable pure-Python path available as a fallback.

Statement of need

The tooling people use to run LLMs locally on Macs — `llama.cpp`, Ollama, LM Studio, and Apple’s own `mlx_lm` (Hannun et al., 2023) — has heavily optimized the compression of model *weights* and the speed of attention, but stores the KV cache at full 16-bit precision by default. Because weight compression is an offline, one-time operation, it does nothing to bound the cache, which is built up token by token at run time and changes with every generation. As inputs grow, unified memory becomes the limiting resource.

KV-cache quantization is an active research area, but the published methods are typically released as standalone research code written for NVIDIA CUDA hardware. They are not directly usable on Apple’s MLX backend, and the custom CUDA kernels they rely on for their reported speedups do not transfer to Apple’s Metal backend. As a result, Mac users and researchers working in the MLX ecosystem have no straightforward, installable way to apply or compare these methods.

VeloxQuant-MLX addresses this need. Its audience is twofold: engineers who run LLMs on Macs and want to fit longer contexts into limited memory, and researchers who need a single

reproducible framework in which KV-cache compression methods can be evaluated on the same Apple-Silicon hardware. The library is installable from PyPI, integrates with an existing `mlx_lm` pipeline in three lines, and ships with tests, documentation, and benchmark scripts that emit machine-readable result files so that every reported number is reproducible.

State of the field

Published KV-cache compression methods cluster into four families, each targeting a different point on the compression–quality–speed trade-off. *Scalar quantization* methods store keys and values at reduced bit-width: KIVI's tuning-free asymmetric 2-bit quantization (Z. Liu & others, 2024), KVQuant's non-uniform, outlier-aware quantization (Hooper & others, 2024), GEAR's quantization-plus-low-rank-residual recipe (Kang & others, 2024), SKVQ's sliding-window channel reordering (Duanmu et al., 2024), and the rotation-and-sketch approaches of TurboQuant (Zandieh & others, 2026) and QJL (Zandieh & others, 2025). *Vector quantization* methods encode keys or values against a codebook: VecInfer (Yao & others, 2025), CommVQ (Li & others, 2025), the calibration-free universal codebook of NSNQuant (Son et al., 2025), and the nearest-neighbor code RaBitQ (Gao & Long, 2024). *Cross-layer and low-rank* methods exploit redundancy between network depths or within a head's dimensions instead of within a single token: PALU's low-rank latent projection (Chang & others, 2025), MiniCache's cross-layer SLERP merge of adjacent layers (A. Liu & others, 2024), and xKV's shared-subspace SVD fit jointly across a layer group (Chang et al., 2025); RateQuant instead allocates bit-width per layer by rate-distortion theory (Anonymous, 2026). A fourth family keeps the cache at full precision but bounds its *length*, evicting or merging the tokens a scoring rule deems least useful: from StreamingLLM's sink-plus-recency window (Xiao et al., 2024) and the cumulative-attention heavy-hitter rule of H2O (Z. Zhang & others, 2023), through SnapKV's prefill-window scoring (Yuan & others, 2025), PyramidKV's and SqueezeAttention's layer-adaptive budgets (Cai & others, 2024; Wang & others, 2024), CaM's merge-instead-of-drop rule (Y. Zhang & others, 2024), and Keyformer's and MorphKV's refinements to the eviction signal itself (Adnan et al., 2024; Ghadia et al., 2025), to query-agnostic reconstruction-reliance scoring as in KVzip (Kim et al., 2025). These are almost all distributed as separate research repositories, each with its own interface and assumptions, and each targeting CUDA.

The build-versus-contribute justification for VeloxQuant-MLX is that no existing package brings these methods to Apple's MLX backend under a common interface, and none could simply be extended to do so: their performance-critical paths assume CUDA kernels, and their APIs are not interoperable. Rather than fork one method, VeloxQuant-MLX re-implements a representative set of them against MLX with a shared cache abstraction, so that they become directly comparable on the same hardware. Including the widely cited KIVI baseline (Z. Liu & others, 2024) is central to this contribution: it is the reference point most KV-cache papers measure against, and its presence lets every other method in the suite be assessed relative to it.

Software design

The library is organized around a small number of abstractions so that adding or swapping a method does not disturb the integration layer. A quantizer registry maps a method name to a Quantizer implementation; a cache factory maps a configuration object to an `mlx_lm`-compatible cache wrapper; and a builder constructs one cache per model layer, which also enables per-layer mixed-precision allocation. Each cache wrapper compresses keys and values inside the standard `update_and_fetch` call and immediately reconstructs them, so that the downstream attention computation always sees ordinary 16-bit tensors and no changes to the model or to `mlx_lm.generate` are required.

A deliberate design trade-off concerns where compression pays off. Because MLX does not

89 expose a sub-byte data type and the published CUDA kernel fusions do not port to Metal,
90 on Apple Silicon the realized benefit of these methods is reduced *memory footprint* rather
91 than increased *throughput*. The library is explicit about this: it reports measured throughput
92 alongside compression so users can see the actual cost, and it provides a pure-MLX reference
93 path for every accelerated kernel so that the fast and slow paths can be checked against
94 each other. The suite deliberately mixes *deterministic* methods — for example KIVI's plain
95 minimum/maximum group quantization, with no learned codebook and no randomness, which
96 is reproducible run to run — with *path-dependent* token-eviction methods whose kept set
97 depends on the order tokens arrive. For the latter the library documents, and tests, the
98 exact reductions that pin each new scoring rule to an existing one: KVzip's single-token
99 reconstruction probe reduces bit-for-bit to the memoryless latest-token TOVA rule, and CaM's
100 cache-merging rule reduces bit-for-bit to H2O's cumulative-attention eviction when its merge
101 step is disabled (`cam_merge="drop"`). Each new eviction method added to the suite is required
102 to state and test such a reduction against an existing method where one exists, so that a
103 method's behavior is characterized rather than asserted.

104 Research impact statement

105 VeloxQuant-MLX is released on PyPI and is accompanied by a documentation site, per-method
106 benchmark scripts, and committed result files for each benchmarked model, so that its claims
107 are reproducible rather than aspirational. Benchmarks are recorded for a range of production
108 4-bit models (including Llama, Qwen, Mistral, Phi, Gemma, and Falcon families) on Apple
109 M-series hardware, with each run writing a machine-readable results file that records the
110 hardware, the sequence lengths, and the realized compression and throughput. The repository
111 also documents negative results explicitly — for example, that the approximate-nearest-neighbor
112 search path of the RaBitQ implementation does not achieve usable retrieval recall — so that
113 users are not misled about scope. By providing a common, reproducible framework on Apple
114 Silicon, the library is intended to lower the barrier to applying and fairly comparing KV-cache
115 compression methods in the growing MLX ecosystem.

116 AI usage disclosure

117 Generative AI tools (Anthropic's Claude) were used during the development of this software, its
118 documentation, and this paper: to help draft and refactor code, to help write documentation
119 and prose, and to help structure benchmarks. All AI-assisted output was reviewed by the
120 author. Correctness was verified through the repository's automated test suite and through
121 benchmark scripts whose numerical results are committed to the repository; the project follows
122 a rule that no performance or compression figure is reported unless it traces to a committed
123 result file produced by a script in the repository, and an internal audit was carried out to
124 remove claims that could not be substantiated from measured data.

125 Acknowledgements

126 VeloxQuant-MLX builds on Apple's MLX framework (Hannun et al., 2023) and re-implements
127 algorithms introduced by the authors of the works cited above; we gratefully acknowledge that
128 prior research, which this library ports to Apple Silicon rather than supersedes. The author
129 received no financial support for this work.

130 References

131 Adnan, M., Arunkumar, A., Jain, G., Nair, P., Soloveychik, I., & Kamath, P. (2024).
132 Keyformer: KV cache reduction through key tokens selection for efficient generative

- inference. *Conference on Machine Learning and Systems (MLSys)*.
- Anonymous. (2026). RateQuant: Mixed-precision KV cache quantization via rate-distortion theory. *arXiv Preprint arXiv:2605.06675*.
- Cai, Z., & others. (2024). PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv Preprint arXiv:2406.02069*.
- Chang, C.-C., Lin, C.-Y., Lin, Y. A., & others. (2025). xKV: Cross-layer KV cache compression via aligned singular vector extraction. *arXiv Preprint arXiv:2503.18893*.
- Chang, C.-C., & others. (2025). Palu: Compressing KV-cache with low-rank projection. *International Conference on Learning Representations (ICLR)*.
- Duanmu, H., Yuan, Z., Li, X., Duan, J., Zhang, X., & Lin, D. (2024). SKVQ: Sliding-window key and value cache quantization for large language models. *Conference on Language Modeling (COLM)*.
- Gao, J., & Long, C. (2024). RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data (SIGMOD)*.
- Ghadia, K. T., Kumar, M., Jain, Y., Nair, N., & Das, P. (2025). Dialogue without limits: Constant-sized KV caches for extended responses in LLMs. *International Conference on Machine Learning (ICML)*.
- Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on apple silicon*.
- Hooper, C., & others. (2024). KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Kang, H., & others. (2024). GEAR: An efficient KV cache compression recipe for near-lossless generative inference. *arXiv Preprint arXiv:2403.05527*.
- Kim, J.-H., Kim, J., Kwon, S., Lee, J. W., Yun, S., & Song, H. O. (2025). KVzip: Query-agnostic KV cache compression with context reconstruction. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Li, J., & others. (2025). CommVQ: Commutative vector quantization for KV cache compression. *arXiv Preprint arXiv:2506.18879*.
- Liu, A., & others. (2024). MiniCache: KV cache compression in depth dimension for large language models. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Liu, Z., & others. (2024). KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *International Conference on Machine Learning (ICML)*.
- Son, D., Choi, E., & Yoo, S. (2025). NSNQuant: A double normalization approach for calibration-free low-bit vector quantization of KV cache. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wang, Z., & others. (2024). SqueezeAttention: 2D management of KV-cache in LLM inference via layer-wise optimal budget. *arXiv Preprint arXiv:2404.04793*.
- Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient streaming language models with attention sinks. *International Conference on Learning Representations (ICLR)*.
- Yao, Z., & others. (2025). VecInfer: Efficient KV cache quantization via vector quantization with outlier-suppressing dual transform. *arXiv Preprint arXiv:2510.06175*.
- Yuan, J., & others. (2025). SnapKV: LLM knows what you are looking for before generation. *International Conference on Learning Representations (ICLR)*.
- Zandieh, A., & others. (2025). QJL: 1-bit quantized JL transform for KV cache quantization.

- 178 *AAAI Conference on Artificial Intelligence*.
- 179 Zandieh, A., & others. (2026). TurboQuant: Online vector quantization with near-optimal
180 distortion rate. *International Conference on Learning Representations (ICLR)*.
- 181 Zhang, Y., & others. (2024). CaM: Cache merging for memory-efficient LLMs inference.
182 *International Conference on Machine Learning (ICML)*.
- 183 Zhang, Z., & others. (2023). H2O: Heavy-hitter oracle for efficient generative inference of
184 large language models. *Advances in Neural Information Processing Systems (NeurIPS)*.

DRAFT